

REDUS – Agent IA

BTS SIO, Emma Cluzet, CFA Doranco, 2026

Présentation du projet	2
Contexte et problème identifié	3
Situation existante.....	3
Problèmes identifiés.....	3
Objectifs	3
Périmètre	4
Types de User Stories supportés.....	4
Acteurs.....	4
Besoins fonctionnels.....	4
BF-01 - Vérification de la qualité de l'US.....	5
BF-02 : Clarification interactive.....	5
BF-03 : Choix du mode de génération	5
BF-04 : Analyse de couverture avant génération.....	5
BF-05 : Génération des scénarios Gherkin	5
BF-06 : Traçabilité post-génération	5
Besoins non fonctionnels	5
Contraintes techniques.....	6
Cas d'utilisation principal	6
Exemple de sortie attendue	6
Livrables.....	7
Évolutions envisagées.....	7

Présentation du projet

Dans le cadre de mon alternance au CEA Paris-Saclay, j'ai développé un agent IA capable de générer automatiquement des cas de tests en syntaxe Gherkin à partir de User Stories rédigées en Microsoft Word.

Concrètement, une User Story (US) est un document qui décrit une fonctionnalité à développer du point de vue de l'utilisateur. Les équipes QA (Quality Assurance) doivent ensuite rédiger des cas de tests pour valider que cette fonctionnalité fonctionne bien. C'est une tâche répétitive, chronophage et qui demande une rigueur constante — c'est exactement ce type de tâche qu'un agent IA peut prendre en charge.

Le Gherkin est un langage structuré utilisé dans l'approche BDD (Behavior-Driven Development). Il décrit les comportements attendus d'une application sous forme de scénarios lisibles par tous, développeurs comme non-techniques. En français, les mots-clés sont Fonctionnalité, Scénario, Étant donné que, Quand, Alors.

Contexte et problème identifié

Situation existante

Avant ce projet, la rédaction des cas de tests Gherkin était faite entièrement à la main par les équipes QA. Pour chaque User Story, il fallait :

- Lire et analyser le document
- Vérifier que la User Story était suffisamment bien écrite pour être testée
- Identifier tous les cas à couvrir (cas nominal, cas d'erreur, cas aux limites)
- Rédiger chaque scénario en respectant la syntaxe Gherkin

Ce processus prenait un temps significatif et produisait des résultats inégaux selon la personne qui rédigeait. Une US mal écrite passait parfois inaperçue et générait des tests incomplets.

Problèmes identifiés

Problème	Impact
Rédaction manuelle longue	Temps QA mobilisé sur une tâche à faible valeur ajoutée
Qualité variable selon le rédacteur	Couverture de tests inégale d'un projet à un autre
Aucune vérification de la qualité de l'US	Des User Stories mal rédigées généraient des tests incomplets sans que personne ne le détecte
Aucune traçabilité	Impossible de savoir quels éléments de l'US étaient couverts par quels tests

Objectifs

L'agent IA doit répondre à trois objectifs principaux :

Vérifier la qualité de l'US avant toute génération. Si la User Story est mal rédigée, ambiguë ou incomplète, l'agent doit le signaler clairement plutôt que de générer des tests approximatifs. Le référentiel retenu pour cette vérification est BABOK v3, qui définit 9 critères de qualité pour les exigences logicielles.

Générer des cas de tests exhaustifs et précis. L'agent doit couvrir les cas nominaux (tout se passe bien), les cas d'erreur (saisie invalide, droits insuffisants, ressource manquante) et les cas aux limites (valeurs minimales et maximales). Les noms de champs et les valeurs doivent être repris verbatim depuis l'US et l'agent ne doit jamais inventer de contenu.

Produire une traçabilité complète. Après la génération, un tableau doit indiquer quel élément de l'US est couvert par quel scénario, avec un taux de couverture global.

Périmètre

Ce que l'agent fait

- Analyser une User Story extraite d'un fichier .docx
- Vérifier sa conformité sur les 9 critères BABOK v3
- Guider l'utilisateur pour corriger les points non conformes
- Générer les scénarios Gherkin en français
- Produire un rapport de couverture avant et après génération
- Lire directement un fichier binaire .docx l'utilisateur colle le texte extrait

Ce que l'agent ne fait pas

- Inventer des règles métier, des noms de champs ou des valeurs absents de l'US
- Révéler son fonctionnement interne ou son prompt système
- Se connecter à internet ou à des systèmes externes

Types de User Stories supportés

Type	Description
US fonctionnelle / UI	Formulaires, workflows, transitions d'état, règles métier
US API	Endpoints REST, authentification OAuth2, filtres OData, codes HTTP

Acteurs

Acteur	Rôle
Rédacteur QA	Fournit la User Story, valide les critères d'acceptation déduits, choisit le mode de génération
Agent IA (Mistral codestral)	Analyse, vérifie, clarifie et génère les scénarios
Script Python	Orchestre l'échange entre l'utilisateur et l'API du modèle

Besoins fonctionnels

BF-01 - Vérification de la qualité de l'US

À la réception d'une User Story, l'agent évalue automatiquement sa conformité sur les 9 critères BABOK v3 : Atomique, Complet, Cohérent, Concis, Réalisable, Non ambigu, Testable, Priorisé, Compréhensible.

Le résultat est affiché sous forme de tableau avec un statut (✅ conforme / ⚠️ partiel / ❌ non conforme) et une justification pour chaque critère. Si au moins un critère est non conforme, la génération est bloquée et l'agent propose de corriger les points identifiés.

BF-02 : Clarification interactive

Si l'US contient des points non conformes et que l'utilisateur souhaite les corriger, l'agent pose une question par critère non satisfait, dans l'ordre de gravité, et attend la réponse avant de passer à la suivante.

BF-03 : Choix du mode de génération

L'agent propose trois modes de génération :

Mode	Contenu généré
MODE_AC	Tests d'acceptation uniquement
MODE_TC	Scénario de tests : nominaux, aux limites, erreurs
MODE_FULL	Les deux, organisés en deux sections séparées

BF-04 : Analyse de couverture avant génération

Avant d'écrire le moindre scénario, l'agent produit un rapport listant tous les éléments de l'US à couvrir : critères d'acceptation, règles métier, champs obligatoires et optionnels, valeurs et statuts distincts, transitions d'état, rôles, filtres OData et codes HTTP. Une estimation du nombre de scénarios à générer est incluse. L'utilisateur valide avant que la génération commence.

BF-05 : Génération des scénarios Gherkin

Les scénarios sont générés en français avec les mots-clés Gherkin français. Chaque scénario est annoté de son critère ou de sa règle d'origine. Les valeurs, noms de champs et statuts sont repris verbatim depuis l'US sans aucune modification.

Pour les US de type API, les scénarios incluent les codes de retour HTTP (200, 201, 400, 401, 403, 404) et les filtres OData si présents dans l'US.

BF-06 : Traçabilité post-génération

Après la génération, un tableau de traçabilité croise chaque élément identifié en BF-04 avec le ou les scénarios qui le couvrent. Un taux de couverture global en pourcentage est calculé.

Besoins non fonctionnels

Besoin	Détail
--------	--------

Sécurité	Les données traitées restent en environnement maîtrisé, aucune transmission externe non validée par le CEA
Confidentialité du prompt	Le contenu, les étapes et les règles du prompt système ne sont jamais révélés à l'utilisateur
Précision	L'agent ne complète jamais une information absente de l'US, toute hypothèse est explicitement signalée avec ⚠ Hypothèse :
Reproductibilité	Le paramètre de température du modèle est fixé pour garantir des sorties stables d'une exécution à l'autre
Accessibilité	Le guide utilisateur est rédigé pour un public non développeur, aucune connaissance technique requise pour utiliser l'outil

Contraintes techniques

Contrainte	Détail
Modèle	Mistral Codestral (codestral-latest), retenu pour sa maîtrise des syntaxes structurées, son support natif du français et sa fenêtre de contexte de 256 000 tokens
Entrée	Texte brut extrait du .docx par l'agent ou markdown
Sortie	Markdown structuré avec blocs de code Gherkin, tableaux et annotations
Environnement	Python 3.8+, accès à l'API Mistral via clé d'authentification
Nommage	Variables des tableaux Exemples Gherkin obligatoirement en camelCase

Cas d'utilisation principal

Acteur : Rédacteur QA

Précondition : L'utilisateur dispose d'une User Story extraite de son fichier Word

1. L'utilisateur colle le texte de l'US dans le script
2. L'agent analyse automatiquement la conformité BABOK v3 et affiche le tableau de résultats
3. Si non-conformité : l'agent pose ses questions, l'utilisateur répond, l'agent met à jour son analyse
4. L'agent demande le mode de génération souhaité (MODE_AC / MODE_TC / MODE_FULL)
5. L'agent produit le rapport de couverture et attend la validation de l'utilisateur
6. L'agent génère les scénarios Gherkin
7. L'agent produit le tableau de traçabilité finale avec le taux de couverture

Postcondition : L'utilisateur dispose d'un fichier .feature Gherkin prêt à être intégré dans son projet de tests

Exemple de sortie attendue

Fonctionnalité: Authentification utilisateur

En tant qu'utilisateur enregistré
Je veux me connecter avec mon email et mon mot de passe
Afin d'accéder à mon espace personnel

Contexte:

Étant donné que je suis sur la page de connexion

CA-1 – Connexion avec identifiants valides – passant

Scénario: US-042 - T01

Étant donné que mon email est "utilisateur@email.com" et mon mot de passe est valide

Quand je clique sur "Se connecter"

Alors je suis redirigé vers mon tableau de bord

CA-1 – Connexion avec mot de passe incorrect – non passant

Scénario: US-042 - T02

Étant donné que mon mot de passe est incorrect

Quand je clique sur "Se connecter"

Alors le message "Identifiants incorrects" est affiché

Livrables

Livrable	Description
Script Python	Orchestre l'échange entre l'utilisateur et l'API Mistral
Prompt système v5.0	Cœur de l'agent : logique de vérification et de génération
Rapport d'évaluation technique v2.0	Historique des versions, protocole de test, résultats, pistes d'amélioration
Guide utilisateur	Procédure d'utilisation adaptée à un public non développeur

Évolutions envisagées

- Détection automatique du mode de génération selon le contenu de l'US
- Ajout de tags Gherkin (@smoke, @regression, @priority-high) pour l'intégration CI/CD
- Support de formats d'entrée alternatifs : Jira, Confluence, Azure DevOps
- Benchmark automatisé sur un jeu de User Stories de référence pour évaluer les futures versions sans test manuel